# Cash Cycle Forecasting & Engagement Timing Agents

Methodology note — Syn Bank Share of Wallet Intelligence Engine (Data School Hackathon, bonus focus area). Companion to cash_cycle_agents.py.

---

## 1. What this solves

The hackathon brief's bonus focus area asks for an agent-based approach that forecasts a client's cash cycles, payment timing, and the best moments for client engagement, and asks that orchestration overhead be kept low. This note documents the two XGBoost agents (forecasting, engagement timing), the Claude-powered briefing agent that sits on top of them, and the incremental-update mechanism that keeps the pipeline cheap to re-run.

## Pipeline overview

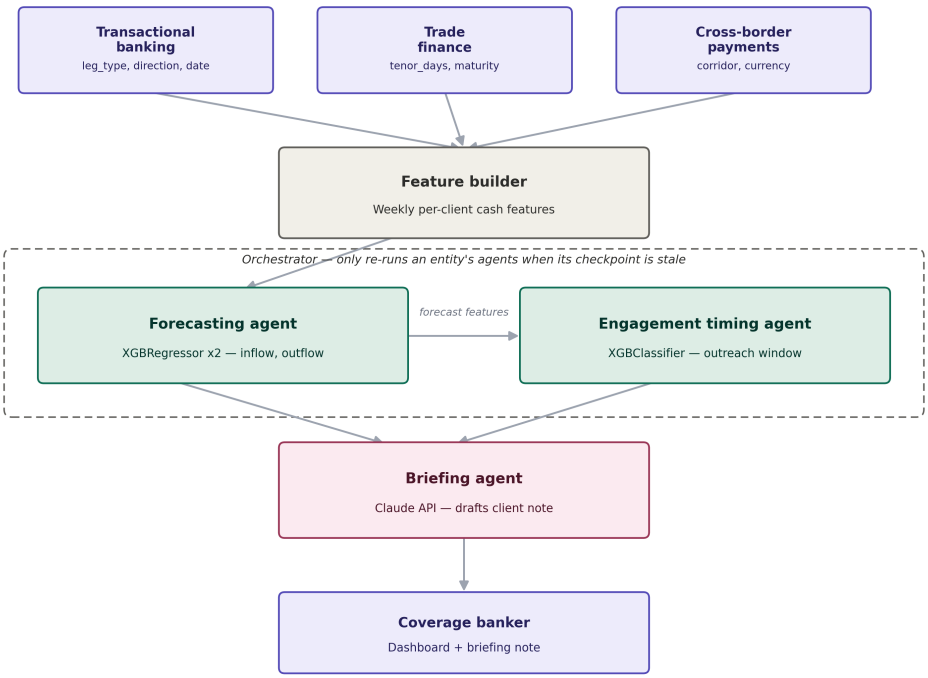


*Figure 1 — three raw data sources feed a shared feature builder; the forecasting and engagement timing agents (XGBoost) sit inside the orchestrator's incremental-update loop; the briefing agent (Claude) turns their structured output into a note for the coverage banker.*

## 2. Why two separate XGBoost agents, not one multi-output model

XGBoost's multi-output mode (multi_strategy) shares tree structure across targets, which helps when targets are the same kind of signal moving together. That is not the case here:

- **Different target types.** The forecasting agent predicts continuous values (next week's collections and payments, in ZAR) — a regression problem. The engagement timing agent predicts a decision (is next week a good outreach window) — a classification problem.

- **A dependency, not a co-prediction.** The engagement timing agent consumes the forecasting agent's output (forecast_net_cash) as an input feature. That chained relationship cannot be expressed as one multi-output model — it needs two models called in sequence.
- **Different retraining cadence.** Cash forecasting should update purely on new transaction data. Engagement scoring logic will change as the bank's outreach playbook evolves (e.g. once real meeting-outcome labels replace the heuristic label used here). Coupling them into one model would force both to retrain together.
- **Separate explainability.** Two focused models mean two clean SHAP / feature-importance views. A judge or banker asking “why did you flag this client” gets a direct answer from one model, not a blended attribution across unrelated targets.

Net: two independent, purpose-built models chained through the orchestrator, rather than one fused model.

## 3. Feature engineering

All three provided CSVs are aggregated to one row per (client, week):

| Source | Signal used |
|---|---|
| transactional_banking.csv | collections (inflow) and supplier_payments (outflow) summed per week; payroll / tax flags for known liquidity drains |
| trade_finance.csv | days until the nearest upcoming instrument maturity (date + tenor_days) and its ZAR value — a forward-looking, non-cyclical signal |
| cross_border_payments.csv | weekly FX/cross-border volume as context for FX-timed cash movements |

On top of these, standard forecasting features are added per client: four weeks of lagged collections/payments, four-week rolling mean and standard deviation of net cash, week-of-month and month (captures payroll/tax seasonality), and a within-30-day trade finance maturity flag. Regression targets are next week's collections and next week's payments, so the model always predicts one week ahead of the most recent completed week.

## 4. Forecasting agent

Two independent XGBRegressor models (300 trees, depth 4, learning rate 0.05) — one for next week's collections, one for next week's payments. Their difference gives forecast_net_cash, the client's expected liquidity position going into the following week.

## 5. Engagement timing agent

A single XGBClassifier (250 trees, depth 3) that takes the same features plus the forecasting agent's three outputs, and scores whether next week is a good outreach window. The label used here is a transparent heuristic — forecast net cash is comfortably positive relative to the client's own recent average, and/or a trade finance instrument matures within 30 days (a concrete reason to call). This is intentionally swappable: once real CRM meeting-outcome data exists, the same feature set can be re-labelled against actual banker follow-through instead.

On the provided synthetic data (July–August 2023, 20 clients with enough weekly history for four lags) a first fit produced a training MAE of roughly R1.9m on inflow and R0.7m on outflow, and an engagement classifier AUC near 1.0 — expected, since the label is a deterministic function of the input features on this small heuristic-labelled sample, and should be treated as a sanity check rather than a generalisation claim. Bonus focus areas note: with only two months of data, four-week lag features consume most of the available history — expect materially more well-calibrated scores once the pipeline runs against a full year or more.

## 6. Briefing agent (Claude)

Only the structured, already-aggregated output of the two XGBoost agents is sent to Claude — never raw client transactions. The prompt asks for a 3-4 sentence client note: expected cash position, whether next week is a good outreach window and why, and one concrete talking point. This is the visible Gen AI component of the submission and directly answers the brief's “client briefing generation” suggestion.

cash_cycle_agents.py — set your key before calling BriefingAgent:

```
ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY", "YOUR_ANTHROPIC_API_KEY_HERE")
```

## 7. Incremental updates — only recompute on new data

A recurring dashboard query should not re-run the full agent chain on every view. Each client has a small checkpoint file recording the latest transaction date the models have already been trained on. On every orchestrator run:

- The orchestrator reads the latest transaction date per client from the raw data.
- If that date has not moved past the client's stored checkpoint, both agents are loaded from disk and reused as-is — no retraining, no feature recomputation for that client.
- If new transactions have landed, only that client's feature rows are rebuilt, and both XGBoost models are **warm-started** — retrained with xgb_model=<previous trees> so boosting continues from the existing trees rather than starting from a cold model.
- The checkpoint is then updated to the new latest date, so the next run only reacts to data that arrived after this one.

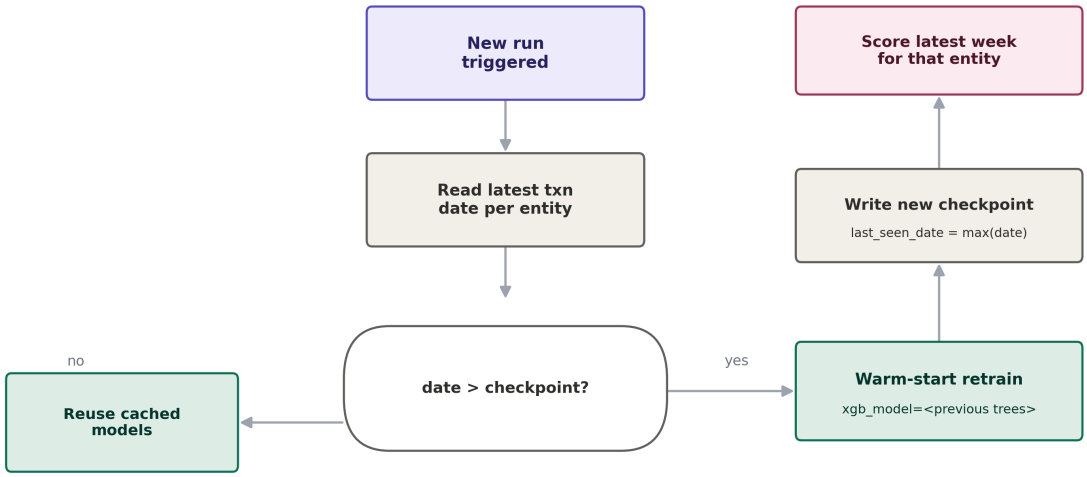


*Figure 2 — the checkpoint check that gates every orchestrator run: cached models are reused whenever a client has no new transactions since the last run.*

This is also the direct answer to the brief's second bonus question, on reducing latency from complex agent hierarchies: the expensive steps (feature rebuild, tree boosting) only run for clients with genuinely new data, and the only per-view LLM call is the lightweight briefing step at the very end of the chain — never the two XGBoost agents, which stay warm in memory or on disk between runs.

## 8. Files

- **cash_cycle_agents.py** — FeatureBuilder, ForecastingAgent, EngagementTimingAgent, BriefingAgent, and the Orchestrator that ties them together with checkpoint-gated updates.
- **data/** — place transactional_banking.csv, trade_finance.csv and cross_border_payments.csv here (paths are read from ./data by default).
- **models/** and **checkpoints/** — created automatically; hold the saved XGBoost model files and per-client checkpoint JSON.

Run `python cash_cycle_agents.py` to fit both agents on the provided data and print the highest-scoring engagement candidates. Pass a list of entity_id values to

Orchestrator.run(generate_briefings_for=[...]) once ANTHROPIC_API_KEY is set, to also print a drafted client note for each.

## 9. Limitations

- Engagement labels are heuristic, not derived from real meeting outcomes — treat the engagement_score as a prioritisation signal, not a probability, until replaced with real labels.
- Only ~2 months of synthetic history is provided; forecasts should stabilise further with more weeks of data and, ideally, a validation split held out by time rather than by row.
- The briefing agent should never receive raw transaction-level data — only the aggregated outputs shown in this note — to keep the Gen AI layer auditable and privacy-safe.